

Mutual Exclusion : **Mutual exclusion** means that only one process can access a resource at a time. It prevents other processes from using the same resource while one is working with it.

This keeps the data consistent and correct, because if multiple processes access the same resource at the same time, data errors can occur.

Multiple Processes

Multiprogramming : The management of multiple processes within a uniprocessor system

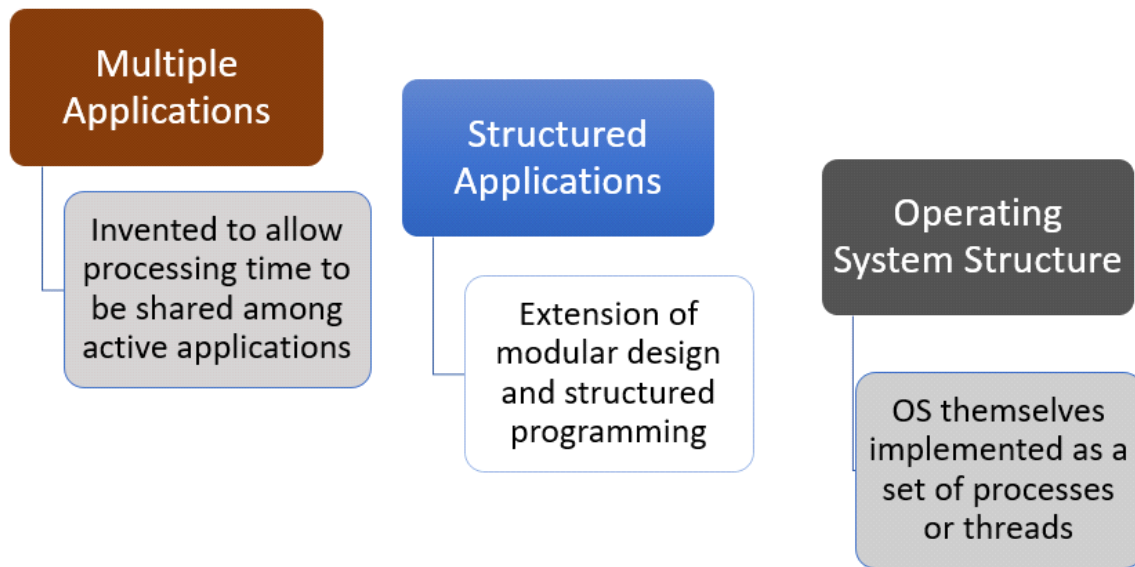
Multiprocessing : The management of multiple processes within a multiprocessor

Distributed Processing :

The management of multiple processes executing on multiple, distributed computer systems.

The recent proliferation of clusters is a prime example of this type of system.

Concurrency arises in three different contexts: Multiple Applications, Structured Algorithms and Operating System Structure. (QUIZ Q.)



Mutual Exclusion: Software Approaches

- Software methods can be applied to concurrent processes on single-processor or multiprocessor systems with shared memory.
- These methods generally assume basic mutual exclusion for memory access.
- Simultaneous accesses to the same memory location are managed by a memory arbiter, although the order of access is not predefined.
- There is no extra support from hardware, operating systems, or programming languages.
- Dijkstra introduced a mutual exclusion algorithm for two processes, initially designed by Dutch mathematician Dekker.
- Solutions are developed step by step, which helps illustrate common bugs in concurrent programming

Principles of Concurrency (Eş zamanlı işleme)

- **Interleaving(Ara Sıralama) and overlapping(Üst Üste Binen İşlemler)**
Both present the same problems (both occurs at multiprocess structure)
 - Uniprocessor – speed of execution of processes cannot be predicted



atomic operation	A function or action implemented as a sequence of one or more instructions that appears to be indivisible; that is, no other process can see an intermediate state or interrupt the operation. The sequence of instruction is guaranteed to execute as a group, or not execute at all, having no visible effect on system state. Atomicity guarantees isolation from concurrent processes.
critical section	A section of code within a process that requires access to shared resources and that must not be executed while another process is in a corresponding section of code.
deadlock	A situation in which two or more processes are unable to proceed because each is waiting for one of the others to do something.
livelock	A situation in which two or more processes continuously change their states in response to changes in the other process(es) without doing any useful work.
mutual exclusion	The requirement that when one process is in a critical section that accesses shared resources, no other process may be in a critical section that accesses any of those shared resources.
race condition	A situation in which multiple threads or processes read and write a shared data item and the final result depends on the relative timing of their execution.
starvation	A situation in which a runnable process is overlooked indefinitely by the scheduler; although it is able to proceed, it is never chosen.

Table 5.1
Some Key
Terms Related
to Concurrency

Concurrency (eşzamanlılık), birden fazla işlemin veya thread'in aynı anda veya üst üste çalıştığı bir durumu ifade eder.

Difficulties of Concurrency

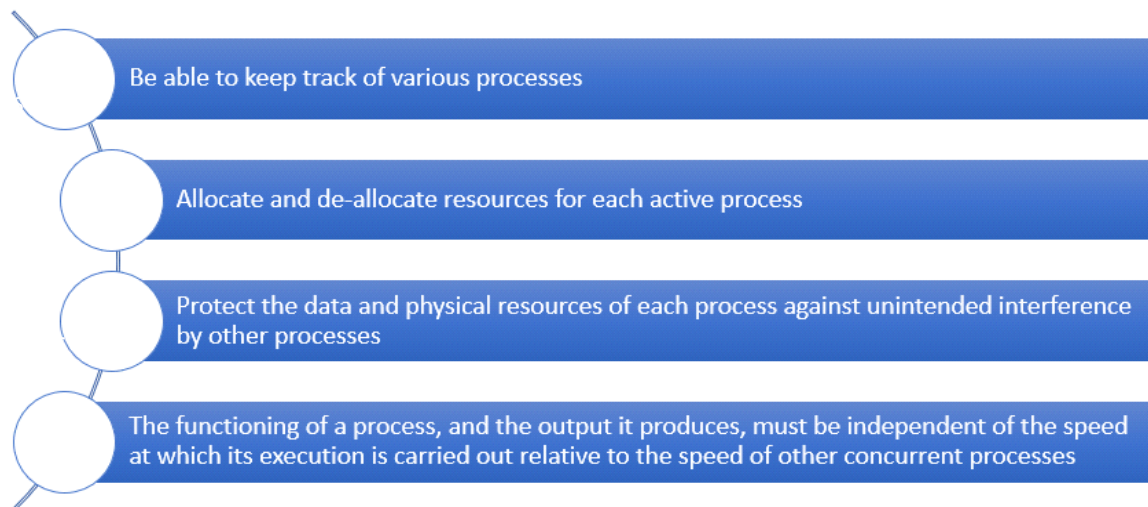
- Sharing of global resources.
- Difficult for the OS to manage the allocation of resources optimally.
- Difficult to locate programming errors as results are not deterministic and reproducible.

Race Condition

- Occurs when multiple processes or threads read and write data items
- The final result depends on the order of execution
 - The "loser" of the race is the process that updates last and will determine the final value of the variable

Operating System Concerns

- Design and management issues raised by the existence of concurrency:
 - The OS must:



10

Resource Competition

- Concurrent processes come into conflict when they are competing for use of the same resource
 - For example: I/O devices, memory, processor time, clock

In the case of competing processes three control problems must be faced:

- The need for mutual exclusion
- Deadlock
- Starvation

! Cooperation Among Processes by Sharing ! (Süreçler Arasında Paylaşım Yoluyla İşbirliği)

Processes work with shared data without knowing about each other directly.

Covers processes that interact with other processes without being explicitly aware of them

Processes use and change shared data, knowing that other processes can do the same.

Processes may use and update the shared data without reference to other processes, but know that other processes may have access to the same data

Thus the processes must cooperate to ensure that the data they share are properly managed

There must be rules to keep shared data correct and safe.

The control mechanisms must ensure the integrity of the shared data

Bc. shared data it might occurs mutual exclusion, deadlock, starvation

Because data are held on resources (devices, memory), the control problems of mutual exclusion, deadlock, and starvation are again present

- The only difference is that data items may be accessed in two different modes, reading and writing, and only writing operations must be mutually exclusive

13

- **Multiple processes work together** towards a common goal, requiring coordination and synchronization.
- **Communication** among processes is essential and is usually done through messages.
- Message sending and receiving can be facilitated by either the programming language or the OS kernel.
- Unlike some other forms of cooperation, mutual exclusion is not necessary here.
- However, issues like deadlock and starvation can still occur.

Requirements for Mutual Exclusion (when there's no mutual exc. we avoid w/ these ways.

- **Enforcement of Mutual Exclusion:** Only one process can enter its critical section at a time for shared resources.
- **Non-Interference:** A halted process must not **disrupt** other processes.
- **No Indefinite Delays:** Processes must not experience deadlock or starvation; access to critical sections cannot be delayed indefinitely.
- **Immediate Access:** If no process is in a critical section, **any** requesting process must be allowed to enter immediately.
- **Independence from Process Speeds:** No guess about speed or number of processes.
- **Finite Critical Section Time:** A process can only stay in its critical section for a limited amount of time.

interleave : iç içe geçirme yeteneği

Mutual Exclusion: Hardware Support

- In a uniprocessor system, processes cannot execute at the same time; they can only run in an interleaved manner.
- A process runs continuously until it calls an OS service or is interrupted.
- To prevent mutual exclusion, it is enough to prevent a process from being interrupted.
- The OS kernel **can do the disable and enable interrupts** for this purpose.

Disadvantages:

- Execution efficiency may significantly decline due to the processor's limited ability to interleave processes.
- This approach will not work in a multiprocessor architecture. Only for uniprocessor system.

- **Compare & Swap Instruction**

- Also called a “**compare and exchange instruction**”
- A compare is made between a memory value and a test value
- If the values are the **same** a swap occurs.
- This operation happens without divided or interrupt.

Special Machine Instruction: Advantages

- Applicable to **any number of processes** on either **a single processor or multiple processors** sharing main memory
- **Simple and easy to verify**
- It can be used to **support multiple critical sections**; each critical section can be defined by its own variable

it works at single processor or multiple processors architecture.

simple and easy to verify.

it supports multiple critical sections.

Special Machine Instruction: Disadvantages

- **Busy-waiting is employed**
 - Thus while a process is waiting for access to a critical section it continues to consume processor time
- **Starvation is possible**
 - When a process leaves a critical section and more than one process is waiting, the selection of a waiting process is arbitrary; some process could indefinitely be denied access
- **Deadlock is possible**

- **Busy-Waiting:** Processes waste CPU time while waiting to access the critical section.
- **Starvation:** Some processes may be indefinitely denied access to the critical section.
- **Deadlock:** Processes may get stuck waiting for each other, causing a deadlock.

SEMAPHORE

A variable that has an integer value upon(üzerinde) which only three operations are defined:

- There is no way to inspect or manipulate semaphores other than these three operations

- 1) A semaphore may be initialized to a nonnegative integer value
- 2) The semWait operation decrements(eksilme) the semaphore value
- 3) The semSignal operation increments(+) the semaphore value

Consequences of Semaphore

There is no way to know before a process decrements a semaphore **whether it will block or not**

There is no way to know which process will continue immediately on a uniprocessor system **when two processes are running concurrently**

You don't know whether another process is waiting so the number of unblocked processes may be **zero or one**

22

A semaphore is an integer value used for communication between processes. There are three atomic operations that can be performed on a semaphore: initializing, decrementing, and incrementing.

- **Decrementing** can block a process if the value is zero.
- **Incrementing** can unblock a process.

Strong/Weak Semaphores

- A queue is used to hold processes waiting on the semaphore
- **Strong Semaphores**
 - This type of semaphore processes transactions added to the queue in FIFO (First In, First Out) order.
- **Weak Semaphores**
 - Does not consider the order in which they are added to the queue and may release transactions randomly.

Producer/Consumer Problem

General Statement:	One or more producers are generating data and placing these in a buffer
	A single consumer is taking items out of the buffer one at a time
	Only one producer or consumer may access the buffer at any one time

The Problem:

Ensure that the producer won't try to add data into the buffer if its full, and that the consumer won't try to remove data from an empty buffer

MONITORS

- **Monitors**: A programming feature that provides similar functionality to semaphores but is easier to manage.
- **Implemented in Languages**: Monitors are used in many programming languages like Concurrent Pascal, Pascal-Plus, Modula-2, Modula-3, and Java.
- **Program Library**: Monitors can also be implemented as a program library.
- **Software Module**: A monitor is a software module that includes one or more procedures, an initialization sequence, and local data.

! Monitor Characteristics !

Local data variables are accessible only by the monitor's procedures and not by any external procedure



Process enters monitor by invoking one of its procedures



Only one process may be executing in the monitor at a time